RetailSoft

Requirements Document

Esther Ajuwon
Ryan Crawford
Robert Salyers
Shirshir Adhikari
Thor Lang

March 20, 2026

V.2

Table Of Contents

1. Introduction / Overview 1
1.1 Purpose 1
1.2 Scope 1
1.2.1 Main Objective 1
1.2.2 Specific Goals 2
1.3 Overview of Document 2

2. System Description 2
2.1 System Context 2
2.2 Target Environment 2
2.3 User Characteristics 3
2.4 Assumptions and Dependencies 3

3. Functional Requirements 3
3.1 Inventory Management 3
3.2 Sales Transactions 4
3.3 Low-Stock Alerts 4
3.4 Reporting and Analytics 4
3.5 Refund and Discount Handling 5

4. Major Subsystems and Components 5
4.1 Inventory Subsystem 5
4.2 Sales Processing Subsystem 5
4.3 Reporting Subsystem 5
4.4 Notification Subsystem 5
4.5 Data Storage Subsystem 6

5. User Interface Specification 6
5.1 GUI Prototype Requirements 6
5.2 Navigation Structure 7
5.3 Report Visualization Requirements 7

6. Non-Functional Requirements (Constraints) 7
6.1 Management Constraints 7
6.2 Technical Constraints 8
6.3 Performance Requirements 8
6.4 Reliability Requirements 8
6.5 Security Requirements 9

6.6 Usability Requirements 9

7. System Evolution / Future Enhancements 9

8. Other Deliverables Required 9

9. Glossary 10

10. References 11

11. UML Diagrams 14

11.1 Use Case Diagram ……..……. 14

11.2 Domain Model Diagram ……...…….. 14

11.3 Class Diagram ……...…….. 15

11.4 Design Diagram ……..…… 17

1. Introduction / Overview

1.1 Purpose

The purpose of this Requirement document is to describe the Small Business Inventory & Sales Management Tool, a lightweight desktop application that assists a small retail business in managing inventory, processing sales transactions, and generating basic analytic reports.

This document provides a structured specification of the system requirements. It serves as a foundational reference for development, validation, and evaluation of the project. It will guide implementation decisions, maintain design consistency, and provide traceability from customer needs to system functionality.

This project aims to demonstrate sound software engineering principles, including structured requirements analysis, domain modeling, validation of business rules, and the design of a CRUD-based application with consistent interaction between business logic and data persistence layers.

1.2 Scope

The scope of this project includes the design and implementation of an offline Windows desktop application for a single-user bike shop environment. The system will focus on inventory tracking, sales processing, low-stock monitoring, and generation of daily and weekly sales reports. The application will operate as a standalone system without integration with external software platforms.

It will emphasize correctness of data handling, accurate aggregation of sales information, and consistency between inventory updates and recorded transactions. The system will not include multi-user role management, external system integration, or advanced security compliance mechanisms, as the customer has specified a single end-user operation on an offline environment.

1.2.1 Main Objective

The main objective is to develop a functional and reliable inventory and sales management application that enables the shop owner to record transactions, and track stock levels, configure alert thresholds, and visualize sales data through automated daily and weekly reports.

1.2.2 Specific Goals

The aim is to accomplish several specific goals that support the main objective. The system will develop accurate inventory tracking mechanisms. The system will implement sales transaction processing with payment categorization. The system will generate timer-based analytical reports in specified visual formats. The system will ensure that refunds and stock updates remain consistent with defined business rules.

Additionally, the customer seeks to apply a prototyping process model to refine the graphical user interface based on customer feedback before full implementation.

1.3 Overview of Document

The outline for this document includes an introduction, system description, functional requirements, user interface specifications, non-functional requirements, and other supporting sections. Each section is organized to present specific aspects of the system in a clear and structured manner.

The document is divided into sections that describe the system environment, user characteristics, major subsystems, and design considerations. Additional sections provide details on constraints, future enhancements, and supporting diagrams used to represent the system.

2. System Description

2.1 System Context

No external applications will be required in order for this application to work. The users of the system will be the employees and manager of the store. The application will receive inventory data and prices from a database which will accept input data from the user. The user will manage inventory and sales through the GUI.

2.2 Target Environment

The application will be an offline Windows application that should work on any Windows system.

During the prototyping phase, the graphical user interface will be designed using Figma. Figma will be used to create interactive mockups of the application screens, including inventory management, sales processing, reporting, and notifications. The prototype will be used to gather customer feedback before implementation begins.

2.3 User Characteristics

The users will not be required to have a significant amount of technical expertise in order to use the application. The users will need to be knowledgeable of the kinds of items their store stocks as well as the prices of those items. An intuitive GUI will be used to navigate the application and manage inventory and sales. The application will be designed to be used in an in-store office environment on a store computer.

2.4 Assumptions and Dependencies

It is assumed that the users will be employees with authorization to access and use the software. The authentication will take place when a user logs in to their computer. The application is dependent upon the database to be filled with stock and price information in order for it to manage inventory and sales.

3. Functional Requirements

There are five main functions of this system, which include inventory management, sales transactions, low-stock alerts, reporting and analytics, and refund and discount handling.

3.1 Inventory Management

The system must allow the customer to create, update, and remove product records within the application. Each product must include relevant attributes appropriate for a bike shop environment, such as product name, category, brand, variation (for example color or model), unit price, and current stock quantity. Products may be classified as bicycles, bicycle parts, apparel, helmets, safety equipment, and other associated merchandise.

Inventory quantities must be updated whenever a sale, refund, or restocking action occurs. The system must keep inventory from going below zero. The customer must be able to manually set a low–stock threshold for each product. The system must produce a low-stock notification when stock drops below the specified threshold.

In addition to managing current stock, the system should allow the customer to enter ordered inventory and projected arrival dates in order to track incoming stock. Local storage and persistence between application sessions are required for all product and inventory data.

3.2 Sales Transactions

The system must support the creation and recording of sales transactions. Each transaction may include several products, and totals must be calculated automatically based on quantity and unit price. The customer must be able to manually apply discounts or promos during checkout before completing the transaction.

For every sale, the system must record the method of payments as either cash, debit, or credit. Completed transactions must be saved in the system for further reporting and aggregation. A transaction may need to be manually re-entered if a system interruption happens before it is finished.

Refund processing must also be supported. When a refund is issued, the system must update the inventory accordingly. Refunded items that are not broken must be returned to stock, whereas broken items must not be restored to available inventory.

3.3 Low-Stock Alerts

The application must monitor inventory levels continuously against manually defined threshold values. When a product's quantity goes below the configured threshold, the system must notify the customer via the application interface. Notification timing must be changeable using a user-controlled timer.

Beyond data entry, low-stock monitoring must function automatically without the need for manual stock inspections.

3.4 Reporting and Analytics

The system must generate sales reports according to defined time schedules set by the customer. A daily sales report must be generated at a user-specified close time. The daily report must be presented as a pie chart, with each segment representing a product category and reflecting total sales for that category during the day.

A weekly sales report must be generated every Monday morning. The weekly report must be presented as a broken bar graph that depicts each day of the preceding week. Each bar must be divided into three payment categories: cash, debit, and credit.

Reports must be readable straight from the application. The basic scope of the project does not involve the export of reports to external file formats.

3.5 Refund and Discount Handling

Manual discounts and promotions must be configurable at the time of sale. Discounts must be factored into final transaction totals before payment is recorded.

The customer must be able to undo previously recorded sales using the refund mechanism. The system must distinguish between broken and unbroken returned objects and adjust inventory levels accordingly. For correct reporting, every refund action needs to be documented in the transaction history.

4. Major Subsystems and Components

4.1 Inventory Subsystem

The inventory subsystem for the application will use a class for inventory stock. Some key attributes for this class will include stock name, stock price, and stock quantity. There will be get and set methods for these attribute values as well as other methods for manipulating the data. The user will manipulate this data through the GUI.

4.2 Sales Processing Subsystem

A sales class will be used for this subsystem, with attributes such as payment method, tax, subtotal, and total. It will need to be able to access data from the inventory class, so the sales class will be a friend to the inventory class.

4.3 Reporting Subsystem

A system for generating sales reports at the end of a business day and week will likely be implemented using another class, and this class will likely need to be a friend of both the sales and inventory classes. It will also generate graphs showing sales and the most popular items sold. Sales reports will likely be generated using Python scripts.

4.4 Notification Subsystem

Notifications informing the user of low stock will likely be implemented in the inventory class as a method. The user should also be notified when a sales report is generated, and this will likely be implemented as a method in the class used for sales reports.

4.5 Data Storage Subsystem

All stock and price data will be stored in a local database which will be accessed by the inventory subsystem. The database will be created and managed using SQLite. This allows the application to function as a lightweight, standalone system without requiring a separate database server.

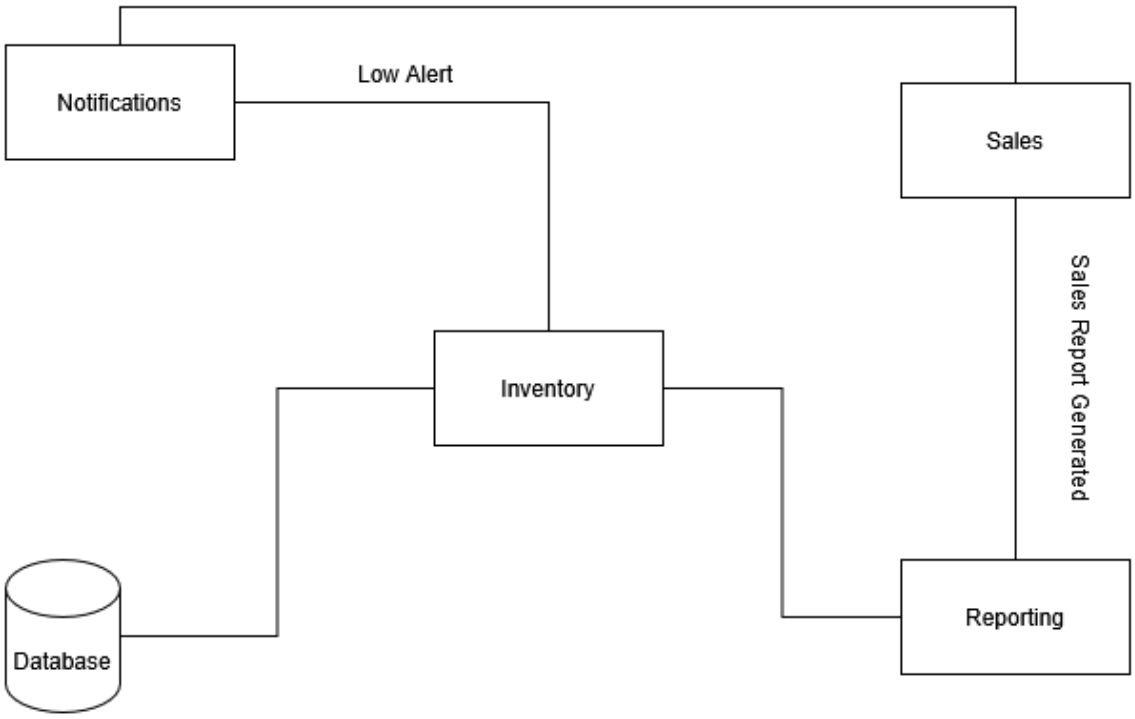


Figure 1 Main Subsystems

5. User Interface Specification

5.1 GUI Prototype Requirements

The GUI prototype must allow the user to access the inventory and view stock quantity and price. The prototype should be able to access a database containing stock information. This prototype showcase will use test data. This data will be delivered with the design documents. The user should be able to enter in values for the low stock quantity threshold for each item, and a basic sales report interface will be shown, using test sales data created by the developers for the purposes of showcasing the interface.

5.2 Navigation Structure

When the application is opened, the user is presented with a main dashboard that serves as the central navigation point of the system. The dashboard provides access to key sections of the application, including inventory management, sales processing, reporting, and low-stock alerts. It also displays summary information such as total sales, monthly performance, and current low-stock items.

The navigation structure allows the user to move between these sections through clearly labeled interface options. Within the inventory management section, the user can view product details, add new products, and perform stock adjustments. Within the sales section, the user can record new sales transactions, view sales orders, and access sales history.

The reporting section provides access to daily and weekly sales reports, report history, and graphical visualizations of sales data. Reports may also include options for exporting or downloading data for further use. The low-stock alerts section allows the user to view items that have fallen below their defined thresholds and manage alert notifications.

The navigation diagram below illustrates the structure and flow between the main components of the system.

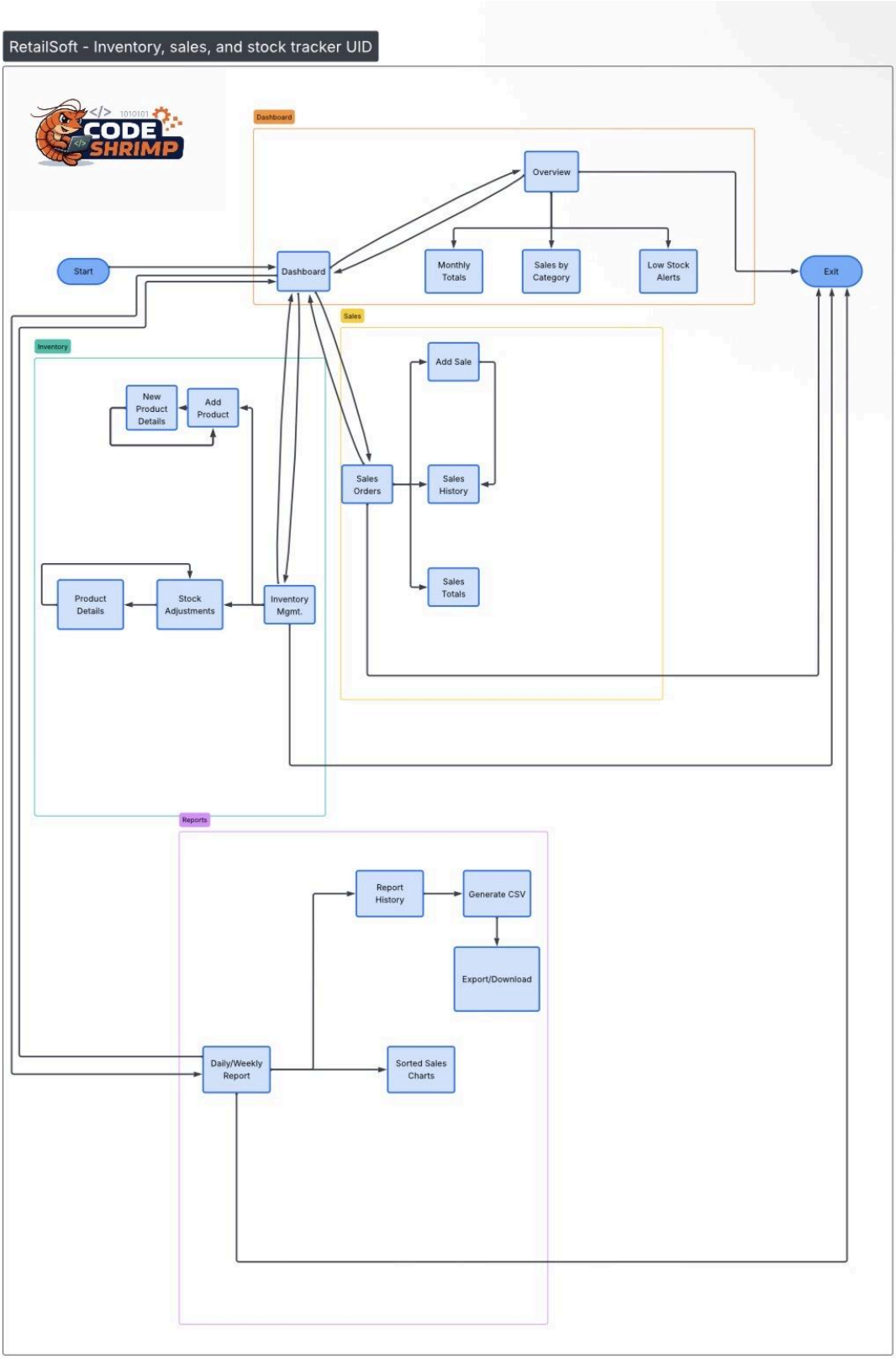


Figure 2: Navigation Diagram of the Inventory and Sales Management System

5.3 Report Visualization Requirements

Daily sales reports must be displayed in the form of a pie chart. Each segment of the pie chart must reflect a product category, such as bicycles, apparel, helmets, shoes, safety equipment, or other relevant items. The size of each section must be proportional to the overall sales value for that category during the business day.

Weekly sales reports must be displayed in the form of a broken bar graph. Each bar must represent one day of the week and be divided into three payment types: cash, debit, and credit. The segmentation must include the total sales made using each payment method on that day.

Daily and weekly reports must be viewable immediately from the application. The basic scope of the project does not need the export of reports to external file formats. The design of the GUI will be delivered with the GUI design documents.

6. Non-Functional Requirements (Constraints)

6.1 Management Constraints

The development process will follow a prototyping model to accommodate early user interface feedback before full implementation. Before final development, the customer must be shown a prototype navigation diagram. After the prototype is approved, the system will be developed using an incremental approach, where system features are implemented and delivered in stages. Further communication during implementation will be minimal, as the customer has requested feedback largely during the prototyping stage and upon final delivery.

The project will be completed within the constraints of academic course schedules and available resources. Data backup capabilities is viewed as a secondary goal and will be developed only if time allows after the basic system functions are completed.

6.2 Technical Constraints

The system must operate as an offline desktop application on a Windows PC. The application must work without internet access and not be dependent on external integrations or cloud-based services. The system must function independently of other business systems and not need to communicate with them.

The system will be implemented using an appropriate programming language such as Python. All data storage must occur locally on the host machine using a relational database management

system such as SQLite. The database must be installed and configured to run locally on the system. The system architecture should maintain separation between user interface components, business logic, and data persistence to ensure maintainability and structural clarity.

Manual data input will be used for product entry, transaction recording, and configuration settings.

6.3 Performance Requirements

Sales transactions must be processed within 2 seconds to ensure smooth operation during

Daily and weekly sales reports must be generated within 5 seconds or less.

Under typical usage circumstances for a small retail store, the system must perform aggregation, stock updates, and report visualization without noticeable performance reduction.

6.4 Reliability Requirements

The system must maintain consistent inventory updates in response to sales and refunds. Inventory level must not fall into negative territory. Transaction data and inventory modifications must always be linked.

Incomplete transactions won't be partially kept in the case of a system crash during a transaction. The user can re-enter the transaction manually. Completed transactions must be kept and accessible between sessions.

Although full automated backup is not part of the primary scope, the system design must allow future implementation of backup functionality.

All inventory records, sales transactions, refund data, stock threshold, and configuration settings must be stored locally and persist between application sessions.

6.5 Security Requirements

Because the system is intended for a single end-user operating on an offline machine, advanced security standards and role-based access control are not required. However, the application must use fundamental data validation procedures to prevent incorrect modification of stored data.

To maintain data correctness, the system must make sure that only legitimate numerical and categorical inputs are accepted for product quantities, pricing, thresholds, and payment methods.

6.6 Usability Requirements

The graphical user interface must be intuitive and organized to allow efficient navigation between inventory management, sale processing, and reporting features. The layout must adhere to the approved navigation prototype provided during the prototyping process.

Daily and weekly reports must present data visually in clearly readable formats. A segmented bar graph categorized by payment type must be shown in the weekly report, and a pie chart grouped by product category must be shown in the daily report.

The user must be able to set report generating times and stock alert thresholds without requiring technical knowledge. For the system to function well, only minimal training should be required.

7. System Evolution / Future Enhancements

In the future, the user interface must be capable of being overhauled to be more visually appealing. Also depending on the customer's feedback to the GUI and preferences. If the application were to be changed to also allow for online use, security authentication must be implemented.

8. Other Deliverables Required

The only deliverables to the customer required are the GUI prototype and the offline application that can run on any Windows system.

9. Glossary

The following terms are used throughout this document to clarify system concepts and maintain consistent terminology.

Inventory: The collection of products currently available for sale within the bike shop.
Product Category: A classification for related things, such as bicycles, helmets, clothing, or beverages.

Stock Threshold: A manually defined minimum quantity for a product that triggers a low-stock alert when reached.
Sales Transaction: A completed purchase record that includes one or more products, payment methods, and calculated totals.
Refund: The reversal of a previously recorded sales transaction, which causes an adjustment to inventory and transaction history.
Broken Item: A product that has been returned but cannot be sold again is not added back into stock.
Unbroken Item: A returned item that has been put back into stock and is still marketable.
Daily Sales Report: A visual pie chart summarizing overall sales by product category over a single business day.
Weekly Sales Report: A segmented bar graph displaying total sales by payment type (cash, debit, or credit) over the course of a week.
Standalone Application: A system that functions on its own without integrating with third-party software or internet resources.
Offline Environment: A setup for the system that can function without internet access.
System Context – The operational environment in which the application functions, including users, hardware, and any related systems.
Target Environment: The application's target platform and operating system, which in this case is an offline Windows desktop environment.
User Characteristics: The anticipated technical expertise, usage context, and skill level of those using the system.
Assumptions – Conditions believed to be true for system operation without formal proof, such as authorized access through the computer login.
Dependencies: External components or situations that the system needs to work properly, such as database availability.
Subsystem: A significant functional element of the application in charge of a certain set of tasks, such processing sales or managing inventory.
Inventory Class: A software class responsible for storing and manipulating product stock data, including name, price, and quantity.
Sales Class: A software class that manages sales transaction data, such as payment method, subtotal, tax, and total.
Reporting Class: A software component used to aggregate sales data and generate graphic reports.
Notification Method: A subsystem function that reports availability or sends out alarms for low supply.
Friend Class: A programming construct that allows one class to access private data members of another class, enabling controlled interaction between subsystems.
Database: A structured data storage system that holds product, inventory, and sales information.

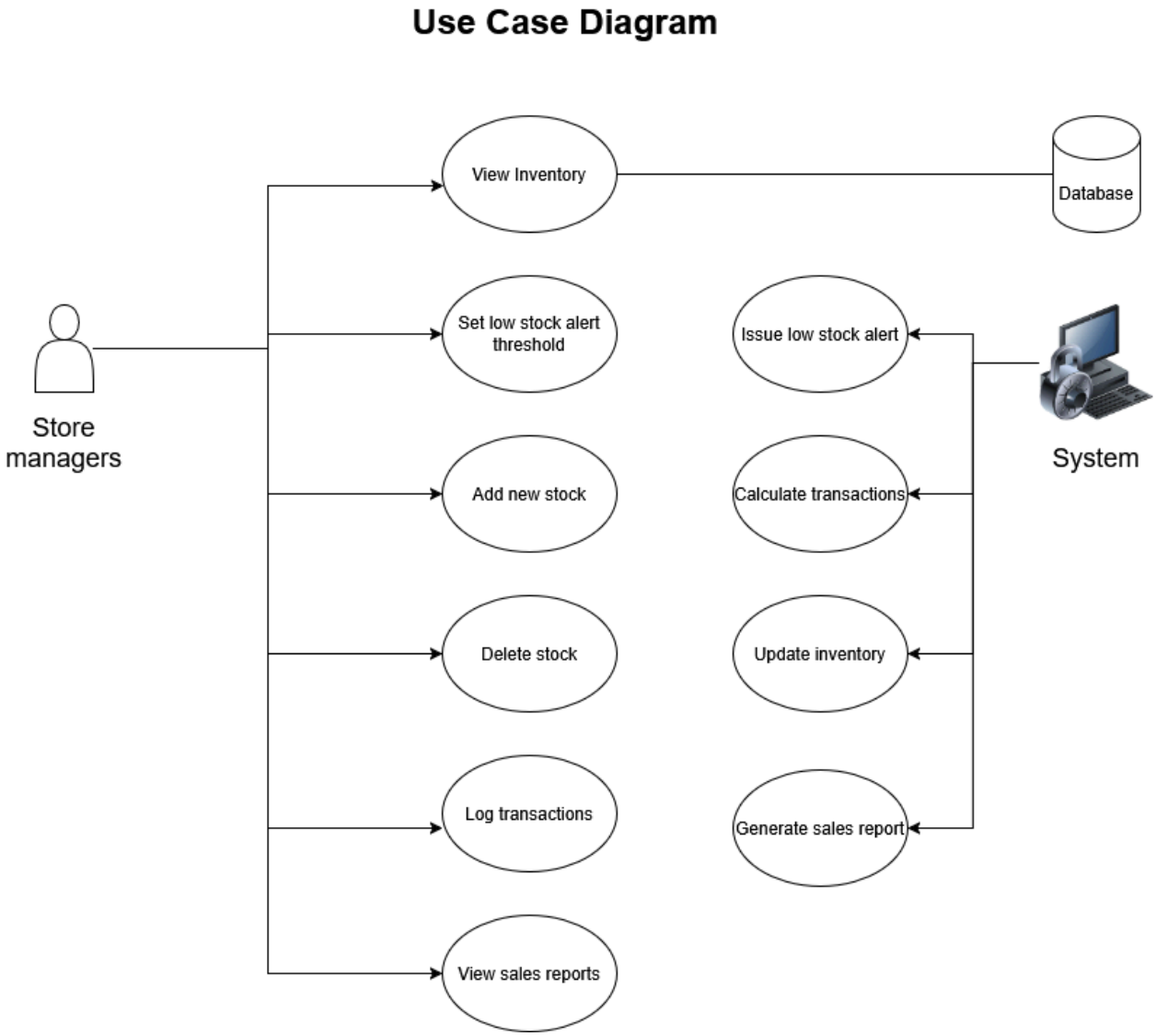
MySQL: The database for the program is created and maintained using a relational database management system.
GUI (Graphical User Interface): The visual interface through which users interact with the system using menus, buttons, and forms.
GUI Prototype: An early visual model of the application interface was utilized to get client feedback prior to final deployment.
Navigation Structure: the arrangement and structure of the screens or pages that let users navigate the application.
Report Visualization: The graphical representation of aggregated sales data using charts such as pie charts or broken bar graphs.
Offline Application: A system that functions without the need for an internet connection.
Standalone System: A program that runs on its own without integrating with other software programs.
Future Enhancement – A potential improvement or feature addition planned for later development phases.

10. References

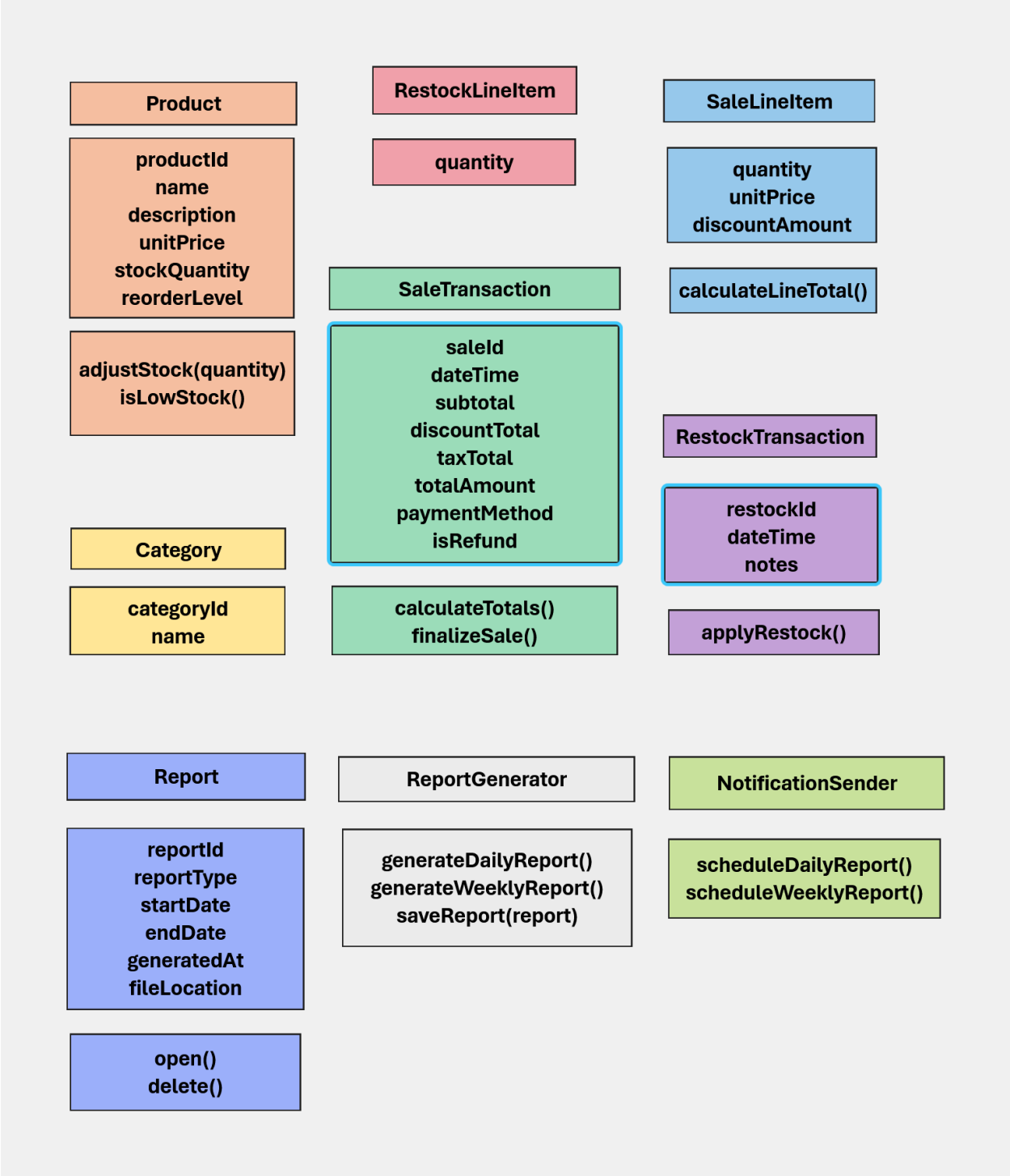
Software Engineering: a Practitioner's Approach by Bruce Maxim and Roger S. Pressman

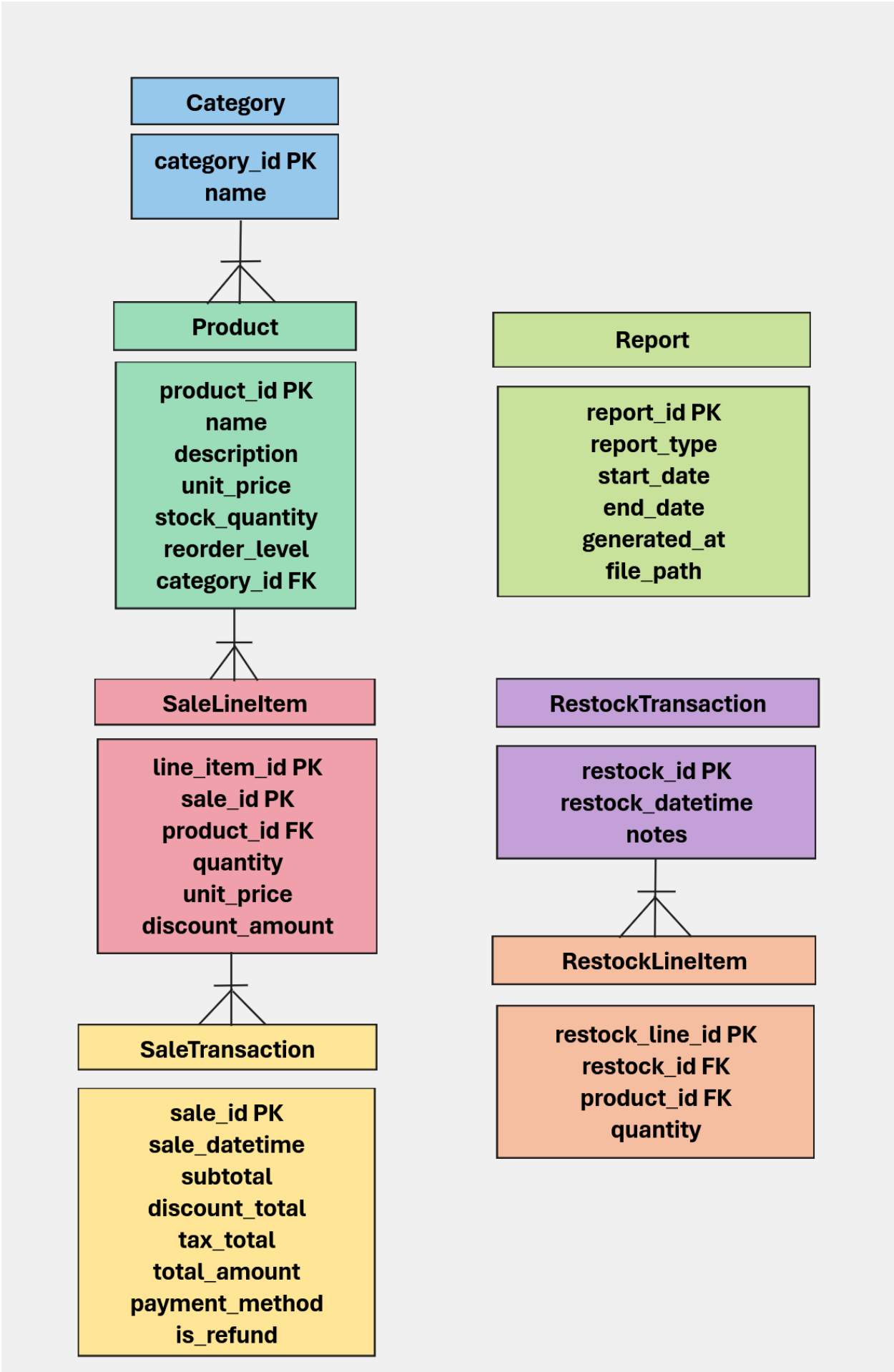
11. UML Diagrams

11.1 Use Case Diagram



11.3 Class Diagrams




Category
category_id PK
name
Product
product_id PK
name
description
unit_price
stock_quantity
reorder_level
category_id FK
SaleLineItem
line_item_id PK
sale_id PK
product_id FK
quantity
unit_price
discount_amount
SaleTransaction
sale_id PK
sale_datetime
subtotal
discount_total
tax_total
total_amount
payment_method
is_refund
Report
report_id PK
report_type
start_date
end_date
generated_at
file_path
RestockTransaction
restock_id PK
restock_datetime
notes
RestockLineItem
restock_line_id PK
restock_id FK
product_id FK
quantity